マルウェア感染を実際に目撃して見たリアルについて

——検証前に整えるべきセキュリティ体制

平木佳介

株式会社サイバーセキュリティクラウド

@k_hirasan

自己紹介

平木 佳介

@k_hirasan

COMPANY

株式会社サイバーセキュリティクラウド

ROLE

TAM（テクニカルアカウントマネージャー）

プロダクト: CloudFastener

DOMAIN

AWS / Azure / Google Cloud セキュリティ

> 今日は **自分が踏んだインシデント** の話をします 😶

本日のアジェンダ

01	インシデントの概要——何が起きたのか	~4min
02	「Marketplace＝安全」という思い込み	~4min
03	調査で判明した侵害の実態	~5min
04	なぜ7日間気付けなかったのか	~3min
05	**検証前に整えるべきセキュリティ体制**	~5min

CHAPTER 01

インシデントの概要

侵害期間 **7日間**

ポートスキャン **2,360回**

GuardDuty **19件 / ALL CRITICAL**

今回の構成と背景

WHAT WE DID

生成 AI アプリ構築プラットフォーム **Dify** を
AWS Marketplace から EC2 へデプロイ

ALB で SSL 終端を設定し
HTTPS でアクセスできるよう構成

目的はあくまで「検証」
"すぐ消すし、どうせ大丈夫だろう"

ENVIRONMENT

AMI	Dify Premium v1.4.2
インスタンス	c5.2xlarge（8 vCPU）
ネットワーク	プライベートサブネット + NAT GW
リージョン	ap-northeast-1（東京）

構成が完了した時点では
すべてが順調に見えました

ある日の深夜

FROM: trustandsafety@support.aws.com
DATE: 深夜 22:24 JST

We've received a report(s) that your AWS resource(s) has been implicated in **Port Scanning**.

このメールをきっかけに調査を開始

AWS Marketplace 経由でデプロイした EC2 が
完全に侵害されていたことが判明

約7日間にわたって悪用されていた

DAMAGE SUMMARY

侵害期間
約 7日間

ポートスキャン
2,360回

GuardDuty 検知
19件 / All Critical

インシデントのタイムライン

1日目 02:04 EC2 インスタンス起動

Dify Premium v1.4.2 デプロイ完了 / ALB + HTTPS 構成

1日目〜2日目 初期侵入（推定）→ マルウェア展開

CVE-2025-55182 を悪用した RCE / CPU: 7-17%（マルウェア展開中）

2日目 06:13 **GuardDuty が検知開始** 19件 / All Critical

通知設定なし → 誰も気づかない

4日目〜9日目 **暗号通貨マイニング全開 + ポートスキャン攻撃継続**

CPU 87-94% 継続 / ネットワーク 100GB/日流出 / 2,360回の攻撃

9日目 22:24 **AWS Abuse Report で初めて気づく**

調査開始 → 10日目 00:30 インスタンス終了

CHAPTER 02

「Marketplace = 安全」という思い込み

CVE-2025-55182

10.0 / 10.0

CVSS スコア / 認証不要 RCE

Dify リリースより 6ヶ月後も脆弱なまま掲載継続

思い込みと現実のギャップ

ASSUMPTION（思い込み）

"AWS Marketplace に掲載されているなら
ある程度セキュリティが
担保されているはずだ"

REALITY（現実）

Marketplace は **流通の場**にすぎない
バージョン管理・脆弱性対応は
ベンダー次第、利用者の責任

項目	詳細
使用バージョン	Dify Premium 1.4.2（2025年6月リリース）
内包脆弱性	CVE-2025-55182 CVSS 10.0 (React2Shell)
CVE 公開日	2025年12月 **← Dify リリースより 6ヶ月後も脆弱なまま掲載継続**

「掲載されているから安全」ではなく、**バージョンの鮮度とベンダーの脆弱性対応速度を自分で確認する**ことが不可欠

CVE-2025-55182 — React2Shell

脆弱性タイプ	**RCE（リモートコード実行）**
CVSS スコア	**10.0 / 10.0**
認証要否	**不要（認証なしで RCE）**
影響コンポーネント	React Server Components
影響バージョン	React 19.0.0 / Next.js 15.2.3

CVE 公開（2025/12）時点でインターネット上に
脆弱な Dify インスタンスが多数稼働
攻撃者はすでに PoC を持っていた

ESTIMATED ATTACK FLOW

攻撃者
↓ CVE-2025-55182 exploit
ALB → EC2 [RCE成功]
↓ Phase 1: 初期侵入
XMRig マイナーダウンロード
ポートスキャンツール設置
Backdoor（Mythic C2）設置
↓ Phase 2: マルウェア展開（2日間）
C&C サーバーと接続・命令受信
↓ Phase 3: 悪意ある活動（約4.5日間）
マイニング + ポートスキャン 2,360回

✅ **現在は修正済み** Dify Premium は現在 **v1.13.2** — CVE-2025-55182 は是正済み

CHAPTER 03

調査で判明した侵害の実態

XMRig マイニング（Monero） CPU 87-94%

Backdoor:EC2/C&CActivity.B Mythic C2

Network OUT 100 GB / 日

GuardDuty 検知 19件 — All Critical

GuardDuty は全部見ていた

侵害から約24時間後（2日目 06:13）から継続的に検知。 全 **19件がすべて Critical レベル**。

検知タイプ	内容
Recon:EC2/PortProbeUnprotectedPort	**2,360回 のポートスキャン**
CryptoCurrency:EC2/BitcoinTool.B	XMRig マイニング（Monero）
CryptoCurrency:Runtime/BitcoinTool.B	コンテナ内でのマイニング検知
UnauthorizedAccess:EC2/TorRelay	Tor リレーノードとの通信
UnauthorizedAccess:EC2/TorClient	Tor エントリーノードとの通信
Backdoor:EC2/C&CActivity.B	**Mythic C2 サーバー通信**

FIRST DETECTION

2日目 06:13

侵害から約24時間後

NOTIFIED VIA

9日目 22:24

AWS Abuse Report で初めて気づく

検知 → 気づくまで

7日間

被害者でありながら、加害者になるという現実

GuardDuty が検知した `Recon:EC2/PortProbeUnprotectedPort` の 2,360件はすべて、侵害された EC2 から**第三者の組織に向けて実行された攻撃**です。

攻撃を受けた組織からすると、
攻撃元はこの AWS アカウント

Abuse Report が届いたのは、
その被害者が AWS に通報したから

インシデントに気づいたきっかけは
他者を攻撃していたから届いた通報

WHAT ACTUALLY HAPPENED

2,360回

他の組織への攻撃（7日間）

「自分が損するだけ」は
根本的に誤った考え方

セキュリティ対策は自分を守るためだけでなく
他者への攻撃加担を防ぐ責任でもある

CPU とネットワークが語る侵害の進行

CPU UTILIZATION — c5.2xlarge (8 vCPU)

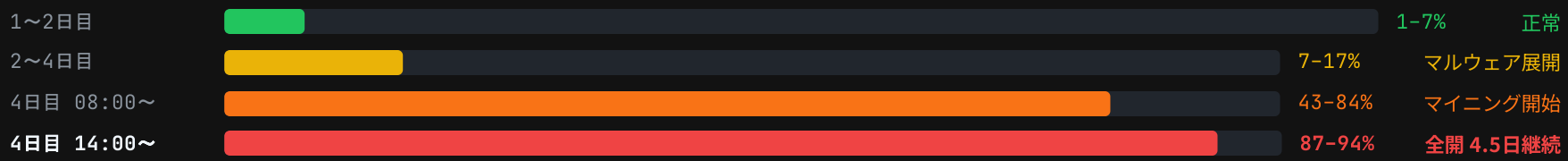
期間	CPU	状態
1日目 〜 2日目	1-7%	正常
2日目 〜 4日目	7-17%	マルウェア展開中
4日目 08:00〜	43-84%	マイニング開始
4日目 14:00〜	**87-94%**	**🔴 全開 (4.5日継続)**

NETWORK OUT

期間	送信量/時	送信量/日
2日目 〜 3日目	1.5 GB	36 GB
4日目 〜 9日目	**4.2 GB**	**100 GB**

💡 **CPU 80%超が30分継続** でアラートがあれば
4日目 14:00 に異常を検知できた
→ 被害期間を7日から約3日に短縮できた

CPU UTILIZATION — VISUAL



ネットワーク構成と「幸運な事実」

NETWORK ARCHITECTURE

```
Internet
│ HTTPS
↓
ALB（パブリックサブネット）
│ HTTP
↓
EC2 [Dify / 侵害済み]（プライベート）
│ Port Scan 2,360回
↓
NAT Gateway（踏み台として悪用）
│ 外部に向けて攻撃
↓
外部ホスト（攻撃対象）

攻撃者 ←→ NAT Gateway ←→ EC2
（C&C通信 / Tor経由）
```

✅ LUCKY FACT

EC2 に IAM インスタンスプロファイルが未設定だった

もし `AdministratorAccess` が付与されていたら…

- 高額な EC2 インスタンスの大量起動
- S3 バケットからの機密データ窃取
- IAM 権限の昇格・横展開
- Bedrock 経由の LLM 不正呼び出し

CloudTrail 確認 →
特権昇格 API コールは一切なし
被害は EC2 内に完全に封じ込め

※ プライベートサブネット配置のため直接アクセス不可だったが、NAT Gateway 経由で外部との通信が可能だったため攻撃の踏み台として悪用された

CHAPTER 04

なぜ7日間
気付けなかったのか

失敗 01 GuardDuty 通知未設定

失敗 02 CloudWatch アラーム未設定

失敗 03 「検証環境だから」という油断

検知から通知まで **7日間**

3つの検知失敗

01 GuardDuty の通知設定が未実施

SNS / EventBridge によるアラートを設定していなかった。19件の Critical 検知がコンソールに溜まったまま、誰も見ていなかった。
「有効化した」≠「通知設定した」

02 CloudWatch アラームの未設定

CPU 90% 台が4.5日継続、ネットワーク送信100GB/日が5日続いても、誰にも通知が飛ばなかった。異常なメトリクスはコンソールに記録されていたが見ていなかった。

03 「検証環境だから」という油断

"細かい監視設定は後でいい"——この判断が7日間という検知遅延の直接の原因。攻撃者にとって検証環境か本番環境かは関係ない。踏み台として使えれば十分。

気づかなかった7日間、EC2 は **第三者の組織に2,360回の攻撃を行い続けていた**

実際のインシデント対応と反省

WHAT WE DID ✅

#	対応内容	時刻
1	GuardDuty 検知内容確認	9日目 22:30
2	影響範囲特定（CloudTrail）	9日目 22:45
3	IAM 権限悪用の有無確認	9日目 23:15
4	EC2 インスタンス終了	10日目 00:30

WHAT WE MISSED ❌（フォレンジック証拠保全）

封じ込め（SG で通信遮断）が先

→ 安全にインターネットから切り離してから調査

EBS スナップショット未取得

→ マルウェア詳細分析ができなかった

メモリダンプ未取得

→ 実行中プロセスの解析不可

VPC Flow Logs 詳細未分析

→ 全通信先の特定ができなかった

⚠ **インスタンス終了は最後の手段**

先に封じ込め・証拠保全 → その後に終了

CHAPTER 05

検証前に整えるべきセキュリティ体制

① AMI バージョン・CVE 確認

② GuardDuty 通知設定

③ IAM 最小権限（NHI）

最低限の 3つの対策

「本番環境だけの話」ではありません——開発・検証環境でも侵害されれば他者への攻撃の踏み台になります。

① Marketplace AMI のバージョン・脆弱性確認

デプロイ前に最新バージョンか確認し、内包ソフトウェアの CVE をベンダーの GitHub・セキュリティアドバイザリで調べる。脆弱なバージョンが侵害の入口になれば他者への攻撃の踏み台になる。

② GuardDuty の通知設定（有効化だけでは不十分）

EventBridge → SNS → Email/Slack で Critical 検知を即座に通知。有効化のみ ≒ 有効化していないのと同じ。これだけで7日間 → 数時間に短縮できた。

③ コンピューティングサービスへの IAM 権限を最小権限に

特に Non-Human Identity（NHI）の権限は十分注意。今回はインスタンスプロファイル未設定が唯一の功績。必要最小限の権限のみを付与する。

対策① バージョン・脆弱性確認の手順

BEFORE DEPLOY CHECKLIST

1. Marketplace の AMI バージョン確認

製品ページのバージョン履歴で最新版かチェック

2. ベンダーの GitHub / Security Advisory 確認

CVE が公開されていないか、パッチは当たっているか

3. 内包コンポーネントの CVE 確認

今回: Dify 1.4.2 → React 19.0.0 → CVE-2025-55182

TODAY'S INCIDENT IN TIMELINE

2025/06 Dify 1.4.2 リリース

2025/12 **CVE-2025-55182 公開**

〜 脆弱なバージョンが Marketplace に掲載継続

2026/02 **インシデント発生**

2026/04 v1.13.2 に更新・CVE 是正済み

CVE 公開から約 **2ヶ月後** に被害発生
その間、Marketplace には脆弱版が掲載され続けていた

対策② GuardDuty — 通知を即時に受け取る

EVENTBRIDGE RULE — EVENT PATTERN

```
{
  "source": ["aws.guardduty"],
  "detail-type": [
    "GuardDuty Finding"
  ],
  "detail": {
    "severity": [{
      "numeric": [">=", 7]
    }]
  }
}
```

NOTIFICATION FLOW

GuardDuty Finding
↓
EventBridge Rule
(severity ≥ 7.0)
↓
SNS Topic
↓
Email / Slack 即時通知 ✅

これだけで
7日間 → 数時間 に短縮できた

BEFORE / AFTER

BEFORE　通知設定なし → 7日後に Abuse Report

7日間

AFTER　EventBridge → SNS → Email/Slack

対策③ IAM 最小権限——NHI の権限管理

もし AdministratorAccess が付与されていたら

💸 高額な EC2 インスタンスの大量起動

攻撃者がコンピューティングリソースを不正利用

🔓 S3 バケットからの機密データ窃取

顧客データ・認証情報・ソースコードの流出

🤖 Bedrock 経由の LLM 不正呼び出し

莫大な推論コストの発生

↗ IAM 権限の昇格・横展開

他のリソースへの侵害拡大

BEST PRACTICE

最小権限の原則

- ✅ 必要な操作のみを許可する権限設計
- ✅ インスタンスプロファイルは必要な場合のみ付与
- ✅ ワイルドカード（`*`）アクションを避ける
- ✅ リソース指定を具体的に絞り込む

特に Non-Human Identity（NHI）に注意

EC2・Lambda・ECS タスクなど人が直接操作しない ID への権限は見落としがち——今回の教訓

まとめ

RISK

Marketplace でも CVSS 10.0 の脆弱性を含む AMI が提供され続ける——**バージョン確認は自分の責任**

TRAP

GuardDuty の通知設定なし = 有効化していないのと同じ——**検証環境も例外ではない**

MINIMUM

IAM 最小権限の原則は必須——**今回はこれだけが救いだった**

IMPACT

侵害された EC2 は7日間で2,360回の攻撃を他の組織に実行——**被害者が加害者になった**

自分のリソースが侵害されても、**「検証環境だし自分が損するだけ」ではない**
侵害されたインスタンスは攻撃者の武器になり、**その矛先は何の関係もない第三者に向かう**

Marketplace 製品であっても、検証環境であっても、セキュリティの基盤は最初から整える

セキュリティ対策は「自分を守る」だけが目的ではない——**他者への攻撃加担を防ぐ責任**でもある

ご清聴ありがとうございました！